Recursive Phased-State Signatures

Problem/Solution Analysis and Mathematical Framework

Research note - May 28, 2026

This document defines a research framework for a recursive phased-state post-quantum signature scheme. It is not a production-ready cryptosystem. The objective is to turn a self-similar key-generation idea into finite mathematics, implementation interfaces, proof obligations, and attack targets.

$$s \to S_\varepsilon \to \{S_v : v \in \mathcal{T}_{D,b}\} \to \{P_v\} \to PK$$

- Conservative base design: SHAKE256 expansion, phase-separated Merkle commitments, and WOTS+/FORS-like leaf signatures.
- Experimental path: replace hash-only expansion with finite geometric recursive maps after the security games are testable.
- Main research question: does self-similar phased key generation improve compression, agility, or batch verification without exploitable cross-scale structure?

1. Problem Analysis

Current post-quantum signature choices create unavoidable tradeoffs. ML-DSA is practical but keeps deployment concentrated in structured lattices. SLH-DSA is conservative and hash-based, but signatures are larger. Falcon/FN-DSA is compact, but implementation is more delicate. Additional candidate families diversify assumptions, yet many rely on complex proof systems, active multivariate assumptions, or novel isogeny machinery.

Target gap

- Compact seed-derived private keys.
- Explicitly separated signing phases for domains, security levels, validity windows, or aggregation layers.
- Recursive, self-similar finite state generation rather than real-valued chaotic maps.
- Public commitments that bind all phases to one public key.
- A credible path to batch verification or aggregation.
- Concrete attack surfaces: related-phase leakage, recurrence recovery, phase confusion, and state reuse.

Non-goal

The framework does not claim that fractal or chaotic behavior is itself a hardness assumption. The cryptographic object must be finite, parameterized, and expressible as security games that can be attacked by the community.

2. Solution Model

A root seed generates a tree of secret states. A phase is a labeled depth or slice of that tree. Each state has a public projection. A public key commits to all phase roots. A signature proves that one valid phase-local secret signed the message and belongs to the committed public key.

$$\Phi = \{\phi_0, \phi_1, \ldots, \phi_D\}$$

$$P_v = \Pi_{\phi_{|v|}}(S_v)$$

$$C_i = \text{MerkleRoot}(\{P_v : |v| = i\})$$

$$PK = H(\text{RPS\_pk} \,\|\, C_0 \,\|\, \cdots \,\|\, C_D)$$

Design rule

Use standard cryptographic expansion for entropy. Let the self-similar idea define the recursive finite structure and phase schedule, not a custom PRG. This keeps the first prototype conservative and makes later geometric variants easier to isolate and analyze.

- RPS-H: conservative hash-based construction with Merkle commitments and one-time/few-time leaf signatures.
- RPS-G: experimental finite-geometric construction where states live in a finite vector space and projections encode a hidden recursive relation.

3. Mathematical Framework

Let lambda be the target security parameter, D the recursion depth, b the branching factor, and T_{D,b} the rooted b-ary tree of depth D. A node is a path v=(j_1,...,j_i) with |v|=i.

$$s \leftarrow_R \{0,1\}^\lambda$$

$$S_\varepsilon = H(\text{RPS\_root} \,\|\, s)$$

$$S_{v\|j} = H(\text{RPS\_child} \,\|\, S_v \,\|\, \phi_i \,\|\, j \,\|\, v)$$

$$P_v = H(\text{RPS\_public} \,\|\, \phi_i \,\|\, v \,\|\, S_v)$$

The expansion is self-similar because the same child relation is used at every scale. Domain strings, phase labels, and canonical path encodings are mandatory; without them, different phases may become related-key domains.

Private/public material

$$SK = s \quad \text{or} \quad SK = (s, \{S_v\}_{v \in C})$$

$$PK = H(\text{RPS\_pk} \,\|\, C_0 \,\|\, C_1 \,\|\, \cdots \,\|\, C_D)$$

4. Algorithms

KeyGen

```
sample s <- {0,1}^lambda

derive S_epsilon and all S_v by recursive expansion

for each phase i:

    L_i = { P_v : |v| = i }

    C_i = MerkleRoot(L_i)

PK = H("RPS-pk" || C_0 || ... || C_D)

return (PK, SK=s)
```

Sign

```
select unused path v with |v| = i

derive S_v from seed s

x_v = H("RPS-ots-secret" || S_v || H(m))

sigma_ots = OTS.Sign(x_v, m)

P_v = H("RPS-public" || phi_i || v || S_v)

pi_v = MerkleAuthPath(P_v, C_i)

return sigma = (i, v, sigma_ots, P_v, pi_v)
```

Verify

```
check |v| == i

check OTS.Verify(P_v, m, sigma_ots)

check MerkleVerify(P_v, pi_v, C_i)

check PK == H("RPS-pk" || C_0 || ... || C_D)

accept only if all checks pass
```

5. Security Games and Proof Sketch

The minimum security model is RPS-EUF-CMA: the adversary gets the public key and signing access for chosen messages and phases, subject to path-use rules. The adversary wins by producing a valid signature for a new message or a new authorized path/message pair.

Cross-scale one-wayness

$$\Pr[A(\{P_v : v \in Q\}) = S_u] \leq 2^{-\lambda} + \varepsilon(\lambda)$$

This captures the key new risk: parent, child, and sibling projections must not make any hidden state easier to recover.

Proof sketch for RPS-H

- If a forgery uses a projection outside the committed phase root, it breaks Merkle collision resistance.
- If it uses a committed but unused projection, it forges the one-time/few-time signature.
- If it changes the phase, explicit phase labels and canonical encodings force a distinct random-oracle domain.
- If it reuses an exposed leaf, the implementation failed state management; a stateless variant must replace the leaf mechanism.

6. Finite-Geometric Variant

The geometric variant replaces hash-only child generation with a recursive finite-space transformation. This should be treated as a second-stage research problem after RPS-H has been implemented and measured.

$$S_v \in \mathbb{F}_q^n$$

$$S_{v\|j} = H_{\mathbb{F}_q^n}(A_{\phi_i,j} S_v + B_{\phi_i,j} + N_{\phi_i,j}(S_v) \bmod q)$$

Hidden Recursive Embedding Problem

Given public projections of a recursively generated finite geometric object, recover a secret state, a hidden path, or a trapdoor relation linking two phases. A candidate is not credible until this problem is parameterized and attacked directly.

- Expected attacks: linearization, Groebner bases, cycle finding, meet-in-the-middle, related-key analysis, and quantum amplitude amplification.
- The design should avoid real-number chaos and floating point; all objects should live over finite fields, rings, graphs, codes, or lattices.
- If the finite-geometric relation collapses into an existing hard problem, it should be presented honestly under that family.

7. Implementation and Evaluation

A minimal implementation should expose four functions and keep the first prototype stateful so leaf reuse is easy to detect during testing.

$$(PK, SK) \leftarrow \mathrm{RPS.KeyGen}(\lambda, D, b)$$

$$\sigma \leftarrow \mathrm{RPS.Sign}(SK, m, i)$$

$$\{0,1\} \leftarrow \mathrm{RPS.Verify}(PK, m, \sigma)$$

$$\{0,1\} \leftarrow \mathrm{RPS.BatchVerify}(PK, \{m_k, \sigma_k\}_k)$$

Evaluation checklist

- Measure public key size, private key size, signature size, signing time, verification time, and batch verification savings.
- Test phase misuse, phase confusion, duplicate leaf use, malformed authentication paths, and non-canonical encodings.
- Run cross-phase statistical checks and structural leakage tests before considering any finite-geometric variant.
- Document exact assumptions: OTS security, Merkle collision resistance, random-oracle domain separation, and state-management constraints.

References: NIST Post-Quantum Cryptography Standardization Project; NIST Round 3 Additional Digital Signature Schemes; NIST IR 8610, 2026.